

Context free Grammar:- also called phrase structure Grammar (PSG) (1) Backus -

Normal form (BNF);

- NLP means:- the capacity of computer software to understand the (NL);

(*) there are variety of languages in the world; each language has its own

structure like $S+V+O$, $S+O+V$

called grammar has $\rightarrow$ certain set of Rules $\rightarrow$ what is allowable
what is not allowable;

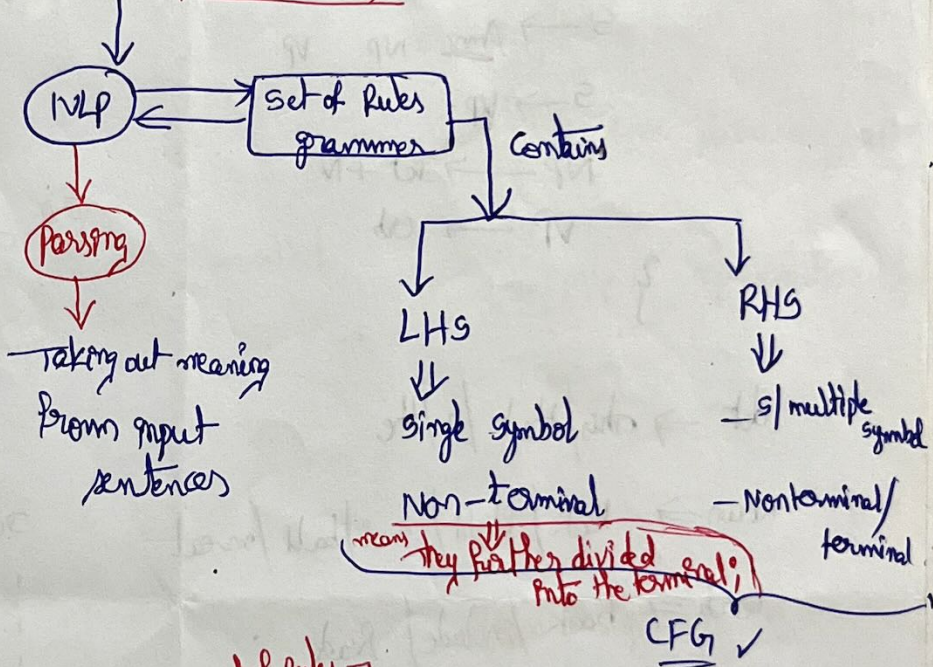
take english:- $S+V+O$

I eat banana

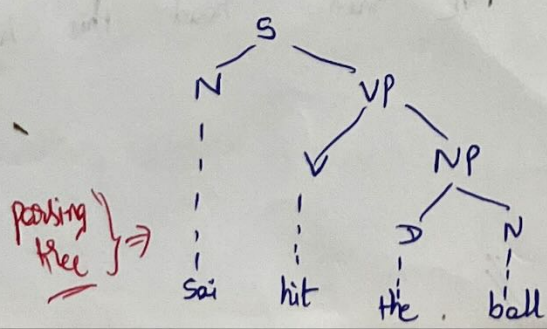
other languages may have $O+S+V$ (may be)

} who will decide this
means Grammar

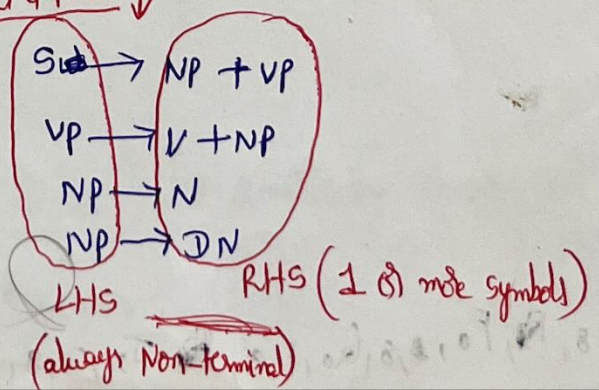
NL as input (sentences)



Example:- sai hit the ball



$\Rightarrow$ set of rules



* Def: CFG is a formal grammar which is used to generate all possible strings in a given formal language;

$$G = (V, T, P, S)$$

G = grammar
 V = Non-Terminal set
 T = Terminal set
 P = Production Rule
 S = Start symbol

also called lexons
 should have must

Rule: $\alpha \rightarrow \beta$
 $\alpha \rightarrow$ single variable
 $\beta \in (V+T)^*$ ✓

(Say) take :-

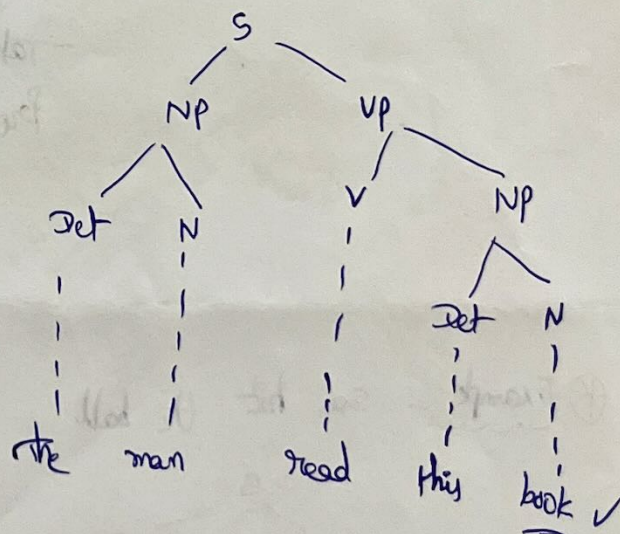
$T = \{ \text{that, this, a, the, book, flight, meal, does, read, sai, ball} \}$

$V = \{ S, NP, VP, Det, verb, \text{aux} \}$ $\Rightarrow$ Non terminal ✓
 auxiliary

$P = \{$
 $S \rightarrow NP \quad VP$
 $S \rightarrow \text{Aux} \quad NP \quad VP$
 $S \rightarrow VP$
 $NP \rightarrow Det + N$
 $VP \rightarrow verb$
 $\}$

phrase = means, may have
 a single word (or) a group
 of words to convey a
 wholistic meaning

(Ex): the man read this book;



det $\Rightarrow$ this/that/a/the

Noun $\Rightarrow$ book/flight/sai/ball/meal

verb $\Rightarrow$ book/include/read

* syntax = noun

syntactic = adj = describes the things related to syntax

meaning, context, log



semantic analysis (meaning)

* feature

syntactic analysis (structure)

1) Focus

How words are arranged (grammar)

what words represent (meaning)

2) Goal

To check grammar & structure
Ensure well formed, valid sentences

ensure logical, understandable content

3) Key Question

'Is this grammatically correct?'

'Does this make sense?'

4) Error type

syntactic errors

semantic errors (logical)

5. Technique

parsing, parse tree, pos tagging

word embeddings, knowledge graph
NER, graph

word sense disambiguation

6. output

parse tree indicating grammatical relationships;

(SA) o/p is. representation of meaning;

'I saw the girl with binoculars'

(Ex): "colorless green ideas slept furiously"

this is syntactically perfect (adj + adj + N + V + adv) but semantically nonsensical;

ideas can't be green & colorless and they don't sleep;

* (Ex) :- ~~Boy~~ ① the milk drank the boy

② I saw a bat ← means mammal
 (a) cricket bat ?

D₅, D₇, 8, E₆, 9, F₀, G₀, 4, 7, H₀, 1, 2, 9, I₉, J₁, 2, 3, 5, 7, 8 Le

1, 2, 3, 8, 13, 22, 25, 27, 29, 30, 31, 32, 34, 35, 37, 39, 40, 43, 45, 47, 49, 50, 52, 54, 55, 56, 57, 61, 62, 65 Le 1, 2, 3, 5, 7

7, 16, 20, 43, 52 Le 3, 5, 7

* L₆, N₀, O₆, P₄, Le 525

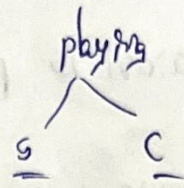
[Faint, mostly illegible handwritten notes and bleed-through from the reverse side of the page.]

* Parsing in NLP :- Breaking a sentence into parts & finding how the words are related to each other. So, parsing is like making a structure of a sentence to understand its grammar & meaning;

Types :-

①

Ex:- Sai playing cricket
(N) (V) (N)
→ acts as root node;



① ~~constitency~~ Constituency Based parsing

② dependency based "

③ top down parsing

④ Bottom up "

⑤ probabilistic parsing

⑥ chart parsing

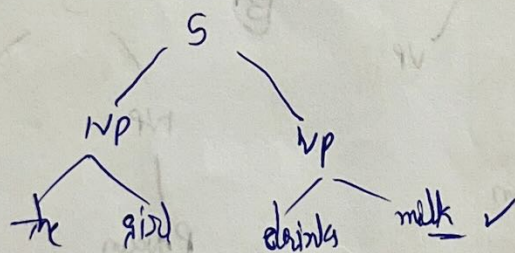
⑦ CYK parser

⑧ Hyper graphs;

* Constituency Based parsing :- (phase structure)

* Breaks into 2-phases ^{NP, VP} ;

Ex:- "the girl drinks milk"



Here Constituency = group of words

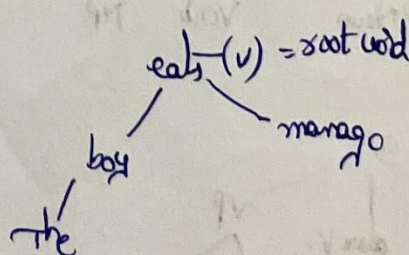
here first find (verb) = drinks ; so

left side part comes under (NP)
Right " " " " (VP) ✓

② dependency based parsing :- V³ Trip

— shows word to word relations (subject, object);

Ex:- the boy eats mango



— It focuses on the relationship between words. Each word dep' on another word.

Here we don't care about phrases; only how words are connected ✓

* Top-down parsing :- start from the root ① ② expand the grammar
 Rules downwards until we reach ~~the~~ ^{each} words;

(Ex):- varavi is playing a game;

$S \rightarrow NP + VP \mid NP + aux V + VP$

$NP \rightarrow Det + nominal\ noun \mid proper\ Noun \mid Noun$

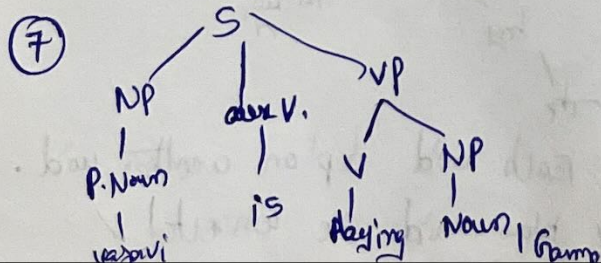
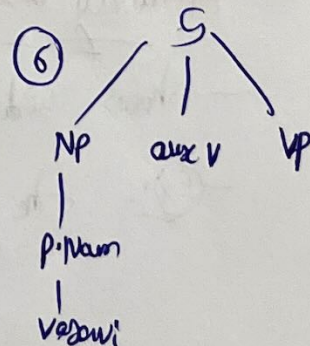
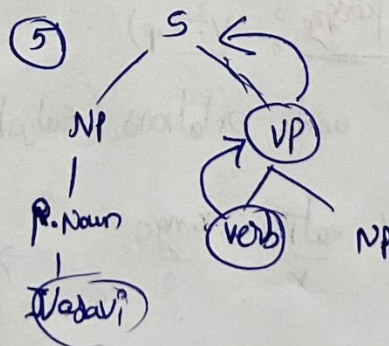
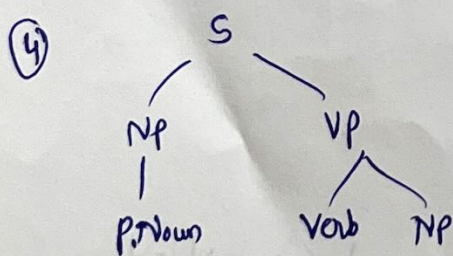
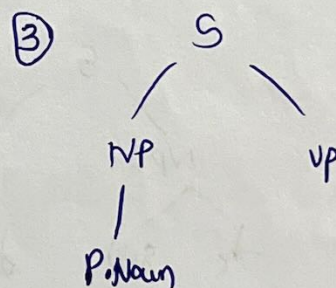
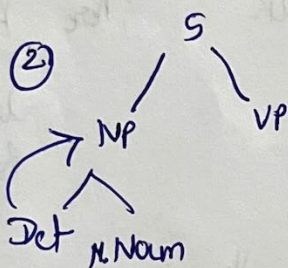
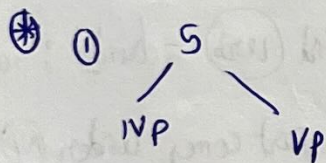
$VP \rightarrow verb + NP \mid V \mid NP$

$P.Noun \rightarrow \text{"Varavi"}$

$Aux\ V \rightarrow is$

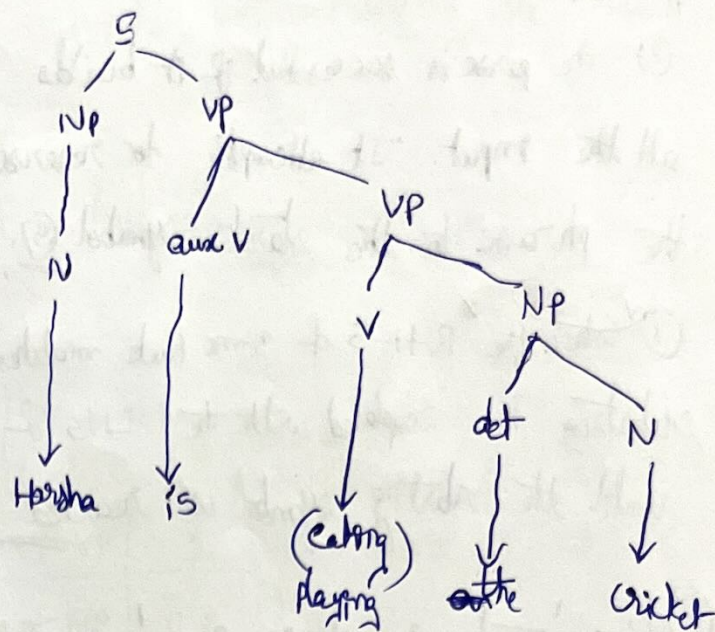
$V \rightarrow playing$

$Noun \rightarrow \text{"Game"}$



Ex ②:- Harsha is playing the cricket;

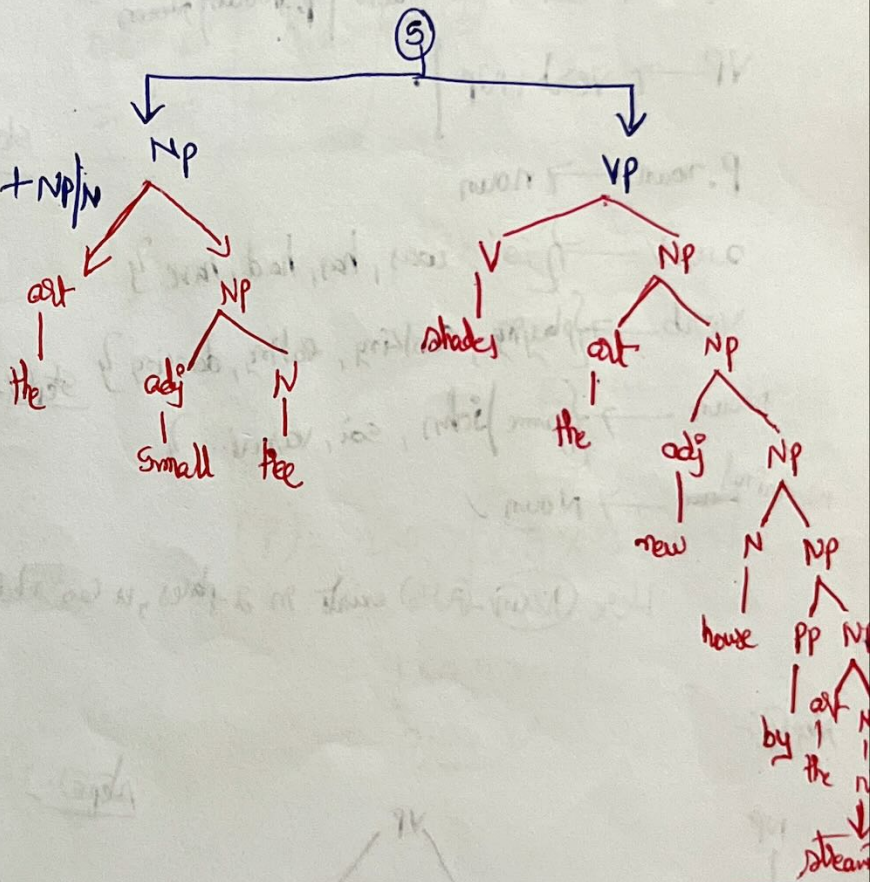
$S \rightarrow NP + VP$
 $NP \rightarrow N \mid \text{art} + N$
 $VP \rightarrow \text{aux} + V$
 $VP \rightarrow V + NP$
 $N \rightarrow \text{Harsha} \mid \text{cricket}$
 $\text{aux} \rightarrow \text{'is'}$
 $V \rightarrow \text{playing}$
 $\text{art} \rightarrow \text{'the'}$



Ex ③ The small tree shades the new house by the stream;

$S \rightarrow NP + VP$
 $NP \rightarrow \text{art} + NP \mid \text{adj} + N$
 $NP \rightarrow \text{adj} + NP \mid N + NP \mid \text{prep} + NP \mid N$
 $VP \rightarrow V + NP$

production rules



② Bottom-up parsing :- Here, the construction of parsetree starts @ the leaves & proceeds towards the root ✓

② The parser is successful if it builds a tree rooted in the start symbol ⑤ that includes all the input. It attempts to reverse the manufacturing process and return the phrase to the start symbol ⑤.

③ ^{v100-14} When the R-H-S of some Rule matches with the substring of the input string, the substring is replaced with the LHS of matched production & the process is repeated until the starting symbol is reached;

(Ex) :- 'John is playing game' $\Rightarrow$ substrings = John, is, playing, game ✓

A :- $S \rightarrow NP + VP \mid NP + auxV + VP$

$NP \rightarrow Det + nominalN \mid P.Noun$ ^{Nominal}

$VP \rightarrow verb + NP$

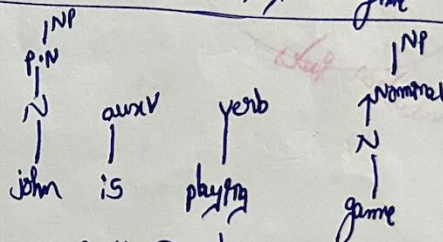
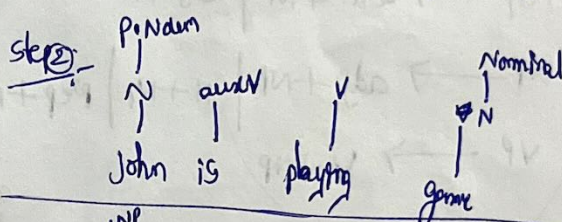
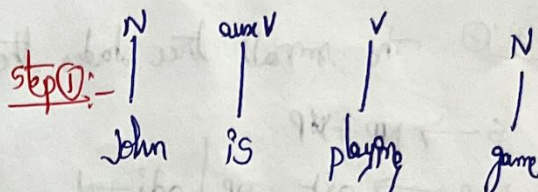
$P.noun \rightarrow noun$

$auxV \rightarrow \{is, was, has, had, have\}$

$verb \rightarrow \{playing, cooking, eating, dancing\}$

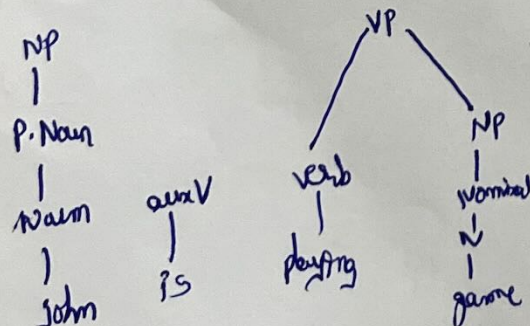
$Noun \rightarrow \{game, John, sai, vasavi, \dots\}$

$Nominal \rightarrow Noun$ ✓

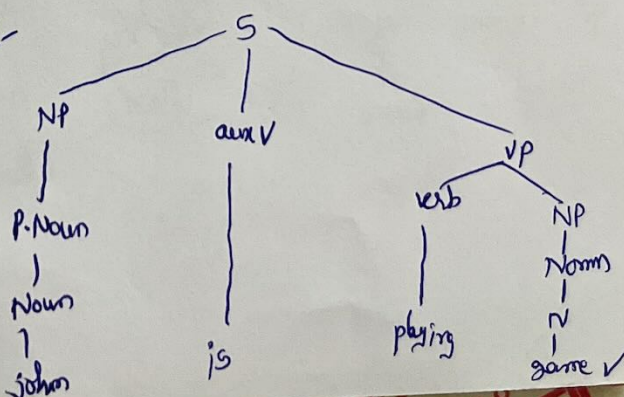


Here (noun) (RHS) exists in 2 places, u can select anyone & finally NP only comes ⑤

step ④ :-



step ⑤ :-



⊛ probabilistic parsing ^{context free grammar} :- (PCFG) :- PCFG defined by pair (G, P) where $G = \text{CFG}$ $P = \text{the func!}$

- probabilistic context free grammars (PCFG's) are used when we want to find the most probable parse structure of the sentence;

- PCFG's are grammar rules, similar to what we have seen ^(CFG), along with probabilities associated with each production rule;

- Ex :- $NP \rightarrow \text{Det} + N(0.5) \mid IV(0.3) \mid PP + N(0.2)$ $= \epsilon P = 1 \checkmark$

ϵ Non-Terminals = 1 ^{i.e. for a non-terminal A, the sum of probabilities of all Rules starting with A must = 1} $\checkmark$

$V_i = \text{sleeps}(1)$

$V_t = \text{saw}(1)$

$NN = \text{man}(0.1), \text{woman}(0.1)$

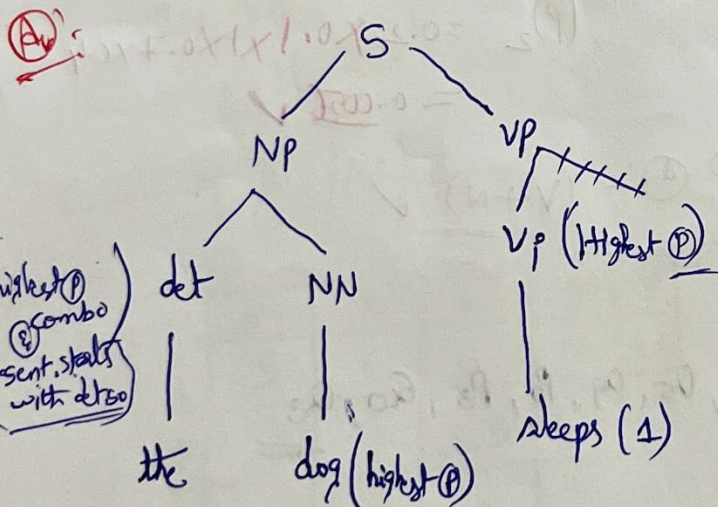
$\text{telephone}(0.3), \text{dog}(0.5)$

$\text{Det} = \text{the}(1)$

$\text{IN} = \text{with}(0.6), \text{in}(0.4)$

S	$\rightarrow NP + VP$	1	①
VP	$V_t + NP$	0.2	
	$VP + PP$	0.2	
	V_i	0.6	
NP	$\text{Det} + NN$	0.8	①
	$NP + PP$	0.2	
PP	$\text{IN} + NP$	1.0	

Ex :- 'the dog sleeps' find probability by drawing parse tree?



$$\begin{aligned}
 (P) &= 1.0 \times [0.8 \times 1 \times 0.5] \times [0.6 \times 1] \\
 &= 1 \times 0.4 \times 0.6 \\
 &= 0.24 \checkmark
 \end{aligned}$$

Right side ↑ Left side ↑

so simply: ⊛ probabilistic parsing is a statistical model that assigns probabilities to possible parse trees & selects the most likely one as the final parse;

⊛ The implementation of statistical parsing based upon PCFG's ✓

1. I like to drink the boy

Ex: $SP \rightarrow NP + VP (0.6) \mid VP (0.4)$

$VP \rightarrow V (0.3) \mid verb + NP (0.7)$

$NP \rightarrow N (1)$

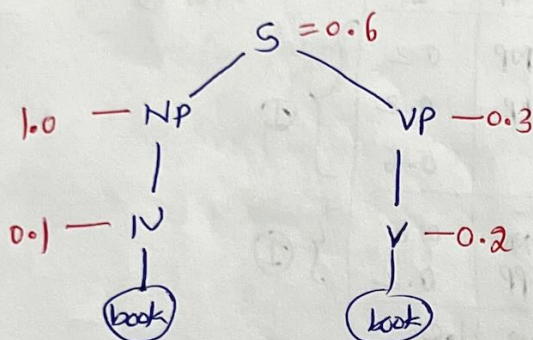
$Noun \rightarrow book (0.1)$

$verb \rightarrow book (0.2)$

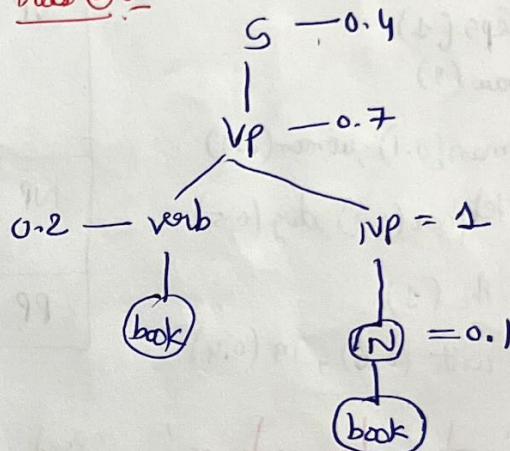
Ex: $S = book book$

① $tree_1$ ② = $\begin{matrix} verb & Noun \\ \uparrow & \uparrow \\ book & book \\ \downarrow & \downarrow \\ N & V \end{matrix}$

trial ① (V, V)



trial ② :-



$(P)_{tree_1} = 0.1 \times 0.2 \times 1 \times 0.3 \times 0.6$
 $= 0.0036 \checkmark$

$(P)_2 = 0.2 \times 0.1 \times 1 \times 0.7 \times 0.4$
 $= 0.0056 \checkmark$

② $\rightarrow P_1$ so Ans $(V + N) \checkmark$

use @ present

$L_0, L_2, L_5, L_6, L_7, M_1, m_5, N_2, N_6, O_5, O_9, P_0, P_3, Q_0, Q_3$

* CYK parsing :- Cocke - Younger - Kasami

① It is a bottom-up parsing method for CFG's;

② Based on dynamic programming concept;

③ It req^d the input grammar to be in 'Chomsky normal form (CNF)';

④ Any context free grammar that doesnot generate the empty string can be rep^d in (CNF) using only production rules of the form;

$$A \rightarrow BC \quad \textcircled{\text{a}} \quad A \rightarrow \underline{a}$$

* CYK parser is efficient for parsing languages that can be represented in CNF, systematically exploring all possible parse trees;

⑥ particularly useful for parsing ambiguous grammars ⑦ ensuring all potential parse

are considered;

* (Ex) :- production Rules

$$S \rightarrow AB/BC$$

$$A \rightarrow BB/a$$

$$B \rightarrow BA/b$$

$$C \rightarrow AC/AA/a$$

Each cell $T(i)(j)$ means
All non-terminals that can
generate the substring from
word i to word $(i+j-1)$

word = a b a b b
1 2 3 4 5

* Triangle shape table

Rows = span length

columns = start position

* Row i means = substring length = i words

column j means = substring start @ word j

① diagonals will be filled with $A \rightarrow \underline{a}$

$\downarrow$
(1,1), (2,2), (3,3), (4,4) & (5,5)

* (1,1) = will be filled with for 1st substring
i.e $a \simeq A(1)C$

(2,2) = b is substring its non-terminal is B

② For (1,2) means a b

$\downarrow$
means (1^a,1) (2^b,2) do concatenate to get ab

$$= (A,C) (B)$$

$$= (AB) (C,B)$$

= S $\Rightarrow$ so only ⑤

Ans :-

0	5	4	3	2	1
1	S	C	S	S	A,C
2		A	B,S	B	
3	C	S	A,C		
4	A	B			
5	B				

11. In the boy

$$(2,3) = \text{means } (2,2) (3,3)$$

$$= (B) (A, C)$$

$$= \underline{BA}, BC$$

$$= \underline{B, S}$$

similarly $(3,4) = S$

$$(4,5) = \underline{A} \checkmark$$

(3) step:- now $(1,3)$ means

1	2	3	(a)	1	2	3
(1,1)	(2,3)			(1,2)	(3,3)	
(A,C)	(B,S)			(S)	(A,C)	
(A,B), (A,S), (C,B), (C,S)				(S,A), (S,C)		
X X X				X X		
<u>S</u> ✓						

finally $(1,3) = \underline{S} \checkmark$

2	3	4	(a)	2	3	4
(2,2)	(3,4)			(2,3)	(4,4)	
B, S				(B,S) (B)		
X				(B,B), (S,B)		
				A X		

finally $(2,4) = \underline{A} \checkmark$

similarly $(3,5) = \underline{C} \checkmark$

(4) step:- $(1,4)$ means

1	2	3	4	1	2	3	4	1	2	3	4
(1,1)	(2,4)			(1,2)	(3,4)			(1,3)	(4,4)		
(A,C)	(A)			(S,S)				(B,S,B)			
(A,A), (C,A)				X				X			
X											

so $(1,4) = \underline{C} \checkmark$

similarly $(2,5) = \underline{S, B} \checkmark$

$$\begin{array}{ccc|c} 1 & 2 & 3 & 4 \ 5 \\ (1,3) & & & (4,5) \\ & \times & & \\ 1 & 2 & 3 & 4 \ 5 \\ & & (1,4) & (5,5) \\ & & & \times \end{array}$$

Ex ①:- $G = \langle T, N, S, R \rangle$

NN = Singular Neuron

NNS = plural

VBG = Gesund verb

ex: - walking, swimming

$$S = S$$

VBJ = present tense, 3rd

$$R = \{ S \rightarrow NP + VP \}$$

Singular verb

$VP \rightarrow VBG + NNS$, $NP \Rightarrow Det + NN$

55 = adjective

$VP \rightarrow VB + VP$, $NP \rightarrow JJ + NNS$

VP $\rightarrow$ VBZ + NP, Det $\rightarrow$ a, NN $\rightarrow$ pilot, VBZ $\rightarrow$ likes,

VBG $\rightarrow$ flying, JJ $\rightarrow$ flying @ NNS = planes;

Statement: - A pilot likes flying planes;
 → means → A pilot likes "flying planes"
 → A pilot likes flying "planes"

Ans:-

	5	4	3	2	1
1					
2					
3					
4					
5					

(d) you can mirror image
also;

Hint :- all diagonals are non-terminals

CNF

Take Rule :-

$A \rightarrow a$ ✓

based on this

$a \rightarrow \text{Det}$
 $\text{pilot} \rightarrow \text{NN}$
 $\text{likes} \rightarrow \text{VBZ}$
 $\text{flying} \rightarrow \text{VBG, JJ}$
 $\text{NNS} \rightarrow \text{planes}$

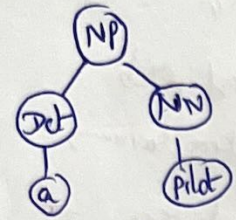
⑥

All inner values will be filled with

$A \rightarrow BC$ ✓

①

a	pilot	likes	flying	planes
Det	NP	---	---	S ₁ , S ₂
	NN	---	---	---
	VBZ	---	VP ₁ , VP ₂	---
	VBG, JJ	VP, NP	NNS	---



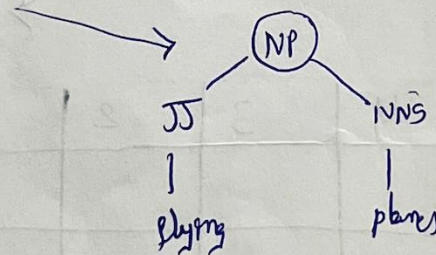
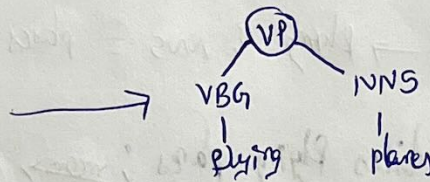
②

02	
01	12
Det	NN
NP (a pilot)	

likewise 13 cell nil

24 cell nil

35	
34	45
VBG, JJ	NNS
VP (flying planes)	
NP (flying planes)	

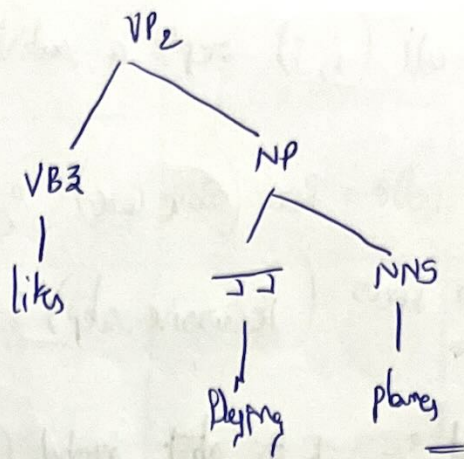
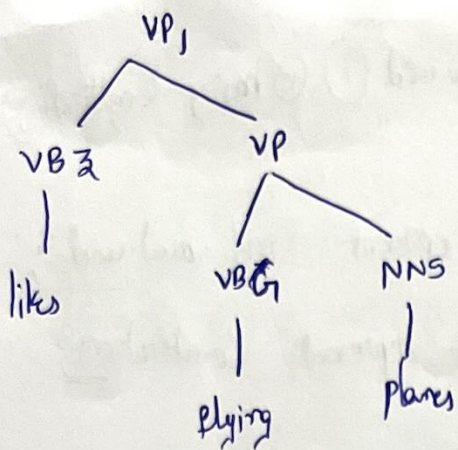


③ stage :-

(0,3)			
(0,1)	(1,3)	(0,2)	(2,3)
Det	nil	NP	VBZ
nil		nil	

(1,4)			
(1,2)	(2,4)	(1,3)	(3,4)
NN	nil	nil	VBG, JJ
nil		nil	

(2,5)			
(2,3)	(3,5)	(2,4)	(4,5)
VBZ	VP, NP	nil	NNS
VP ₁ (likes, VP)		nil	
VP ₂ (likes, NP)			



Stage 4:- anyone of the combo is (nil) total = nil

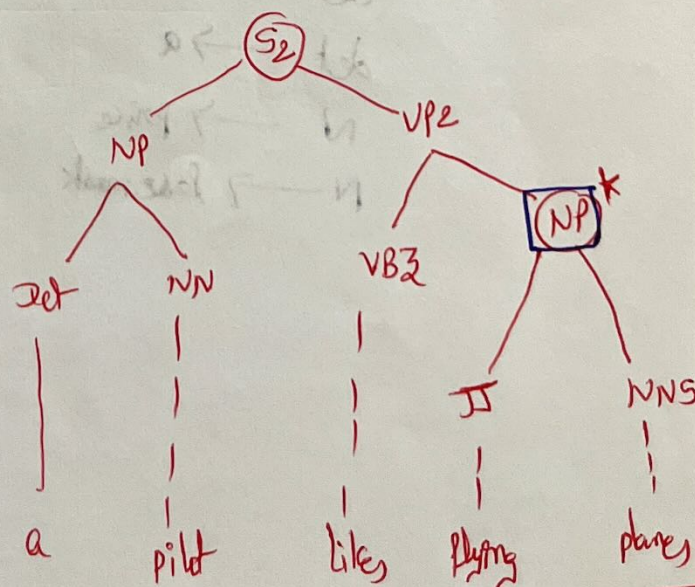
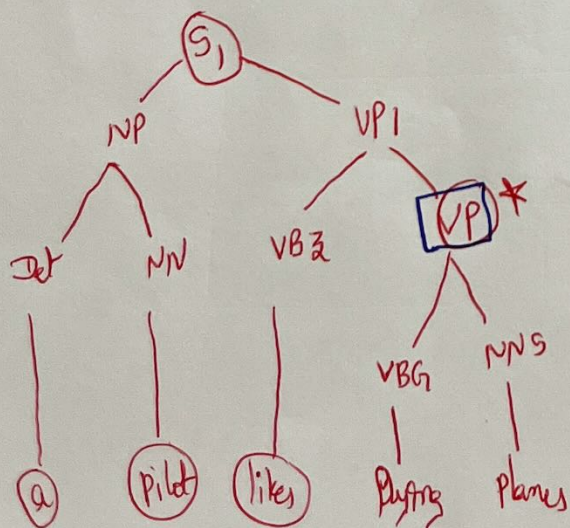
(0,4)					
(01)	(14)	(02)	(24)	(03)	(34)
det	nil	NP	nil	nil	VBG, JJ
nil		nil		nil	

(15)					
(12)	(25)	(13)	(35)	(14)	(45)
NP	VP	nil	VP, NP	nil	NNS
nil		nil		nil	

Stage 5:-

(05)					
(01)	(15)	(02)	(25)	(03)	(35)
det	nil	NP	VP1, VP2	nil	VP, NP
nil		$S_1(NP, VP_1)$ $S_2(NP, VP_2)$		nil	
				NNS	

* indicates 2 parse trees



* now with probabilistic cyk we will rank the S_1 & S_2 whichever is higher that is S_1

- ⊗ Every cell (i, j) rep's a substring starting at i and ending at j
- ⊗ The Bottom Row (Base case) :- These cells represent individual words;
- ⊗ Higher Rows (Recursive step) :- These cells represent combinations;

The Goal :- If the start symbol (S) appears in the very top cell, ~~then~~ then the sentence is grammatically correct ✓

- adv :-
- ① Guaranteed completeness
 - ② Efficient for CFG's
 - ③ Dynamic prog. approach
 - ④ Handles ambiguity effectively; (based on CYK)
 - ⑤ Foundation for modern parsers!

PCYK :- probabilistic CYK or cyk :-

Ex :- the price includes a face mask;

⊗ Rules :-

production Rule	probability
$S \rightarrow NP + VP$	0.8
$NP \rightarrow Det + N$	0.3
$VP \rightarrow V + NP$	0.2
$V \rightarrow includes$	0.05
$det \rightarrow the$	0.4
$det \rightarrow a$	0.4
$N \rightarrow price$	0.01
$N \rightarrow face\ mask$	0.02

The	price	includes	a	Face mask
Det (0.4)	NP (0.3 × 0.4 × 0.01) = 0.0012	X	X	S (0.8 × 0.0012 × 0.000024) = 2.304 × 10 ⁻⁸ ✓
	N (0.01)	X	X	X
	V (0.05)	X	X	VP (0.2 × 0.05 × 0.0024) = 0.000024
			Det (0.4)	NP (0.3 × 0.4 × 0.02) = 0.0024
				N (0.02)

* Shallow parsing :- It focuses only on local struc, not the complete sentence grammar.

* also called chunking (i) light parsing (ii) partial parsing;

(2) This is an (NLP) technique that identifies & groups words into non-overlapping phrase-level units (VP, NP, PP) without constructing a complete syntactic parse tree;

(3) $NP \rightarrow Det + N \mid Det + JJ^* + NN$
 $\downarrow$
 zero (s) more adjectives

Ex:- the quick brown fox
 $\downarrow \quad \downarrow \quad \downarrow \quad \downarrow$
 Det JJ JJ NN (NP)

$VP \rightarrow MD + V + Aux V + VBG$
 $\downarrow \quad \downarrow \quad \downarrow$
 modal auxiliary verb gerund verb (swimming, walking)

$PP \rightarrow IN + NP$
 $\downarrow$
 preposition $\rightarrow$ Noun phrase $\Rightarrow$ Ex:- on the mat
 $\downarrow \quad \downarrow \quad \downarrow$
 IN Det N
 $\downarrow$
 + NP
 $\downarrow$
 PP $\checkmark$

(4) Shallow parsing freqly uses (BIOES) or IOB tagging to define boundaries for (ML) models
 B = Beginning :- Indicates 1st token of a named entity;
 I = Inside :- Indicates that the token is a part of an entity but not 1st one;
 O = outside :- " that the token doesnot belongs to any entity;

Ex:- "Harsha went to paris"
 $\downarrow \quad \downarrow \quad \downarrow \quad \downarrow$
 B-person O O B-loc

* Core principles of production Rules :-

- ① Non-recursive — A (NP) can't contain another (NP)
- ② Non-overlapping — A word can belong to at most one chunk;
- ③ Context-Independent — Rules mostly rely on tags of adjacent words, not the entire sentence
- ④ POS tag dependent — Rules are written based on Pos tags;
- ⑤ Chunks are non-exhaustive

→ converting ^{nominal} categorical data into numbers;

* One-hot Encoding :- NLP models work with numbers, not raw text; (string)

* vectorization = converting text into numerical format;

Common methods :- ① one-hot encoding ② Bag of words ③ TF-IDF

④ Word Embeddings (word2vec, GloVe etc.) (Term Frequency - Inverse document freq)

Types of encoding :-

- ① Label encoding
- ② one-hot " ⑥ mean encoding
- ③ Ordinal " ⑦ freq "
- ④ k-fold Target "
- ⑤ Binary " ⑧ bag "

* one-hot represents :- words as binary vectors;

vector length = size of the vocabulary;

one position = hot (1) ; rest are cold (0);

- very sparse but simple & effective for small vocabularies;

* categorical data → one-hot encoding
 → Nominal :- Not having intrinsic order; Ex: male, female
 → Ordinal :- having intrinsic order; Ex: good, bad, excellent
 ↓
 (we ordinal encoding)

Ex:- (3)-categories :- (Y, B, R)

<u>color</u>	<u>Target</u>		<u>color_Y</u>	<u>color_B</u>	<u>color_R</u>	<u>Target</u>
Y	0		1	0	0	0
Y	1		1	0	0	1
B	1		0	1	0	1
Y	1		1	0	0	1
R	1	⇒	0	0	1	1
Y	0		1	0	0	0
R	1		0	0	1	1
R	0		0	0	1	1
Y	1		1	0	0	1
B	<u>0</u>		0	1	0	<u>0</u>

here $\begin{bmatrix} 1 & 0 & 0 \end{bmatrix}$ = Yellow

$\begin{bmatrix} 0 & 1 & 0 \end{bmatrix}$ = Blue

$\begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$ = Red

lets say If you have 50 diff categories then we will build 50's columns;

Ex (MLP):	Text	O/P
s ₁	my name is sai, hanuman sai	1
s ₂	sai is good	0
s ₃	hanuman sai is coming	1

(Total words = 13)

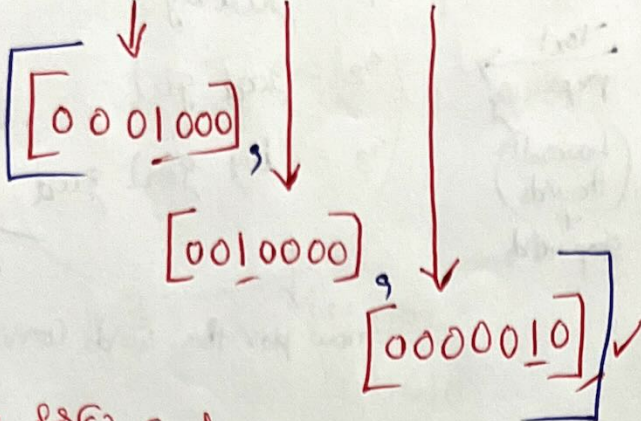
— pool all the words & put them in a (one bag) = Corpus C

— take unique words = vocabulary (V) ; here (V=7)

my
name
is
sai
hanuman
good coming

$V =$ my name is sai hanuman good coming

now $S_2 =$ sai is good



likewise for S_1, S_3 also ✓

vectorized text

it looks like a matrix

size is (3×7)

no. of words
in
a sentence

adv:- (1) easy to implement with python (sklearn, one-hot encoder (lib))

(2) Intuition easy;

Disadv:- (1) generates sparse matrix; which consists of predominate of zeros; which leads to overfitting;

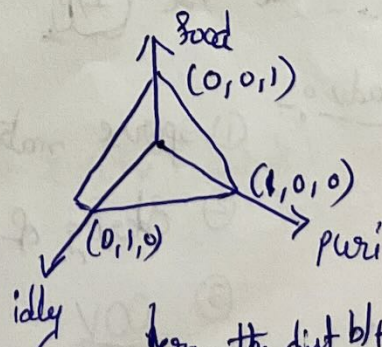
(2) generally for ML algorithm, we need to fix size of $\mathbb{R}^p$;

observe above examples size ✓

(3) NO semantic meaning is getting captured;

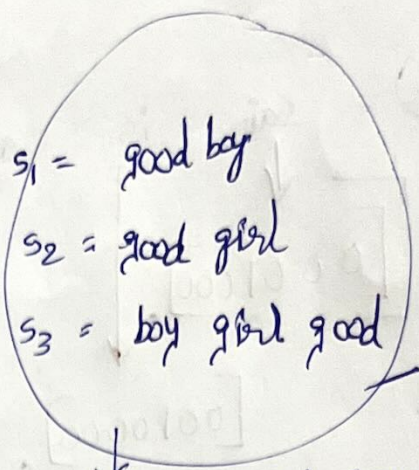
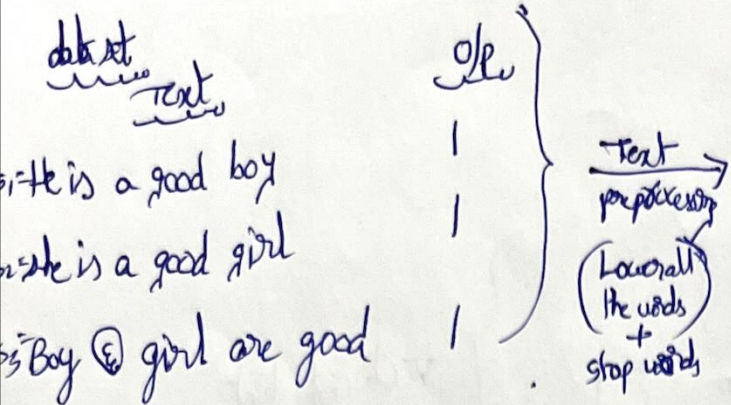
(4) (OOV) out of vocabulary ✓

means if the word not in vocabulary then encoding fails ✓



here, the dist b/w any 2 words is almost same that means there is no disguised not occurring

② Bag of Words (BOW) :-



now how this words conv. to numbers

with BOW?

① vocabulary :- (good, boy, girl)

② frequency :-

good - 3

boy - 2

girl - 2

	like v			
	good	boy	girl	o/p
s_1	1	1	0	1
s_2	1	0	1	1
s_3	1	1	1	1

③ Binary Bow :- only {0 (or) 1} Vs BOW

↓

{Count of word will get updated based on freq}

④ advantages :- ① simple & Intuition;

② fixed size (I/p) so we can apply ML algorithms

⑤ Disadv :-

① sparse matrices; so overfitting happens;

② ordering of the words is getting changed;

③ COV

④ semantic meaning is not getting captured;

(another explanation)

V = my name is Sai hanuman good coming

S_1	=	1	1	1	2	1	0	0
S_2	=	0	0	1	1	0	1	0
S_3	=	0	0	1	1	1	0	1

so finally $S_1 = 1112100$
 $S_2 = 0011010$
 $S_3 = 0011101$

Here we will get uniform size of vectors;

↓
 this idea will helps in
 Bag of n-grams @ TF-IDF

③ Bag of N-grams: - same as bag of words with little changes;

take $n=1 \rightarrow$ unigram
 $n=2 \rightarrow$ Bi "
 $n=3 \rightarrow$ Tri "

we have ③ sentences @ total words = 13
 so total Bigrams = 10

* In (BoW) each word is behave as one-unit; but Bag of bigram, each word have adjacent word similarly trigram @ etc...

so Corpus: - my name
 (bigram) name is

is Sai
 Sai hanuman

hanuman Sai

Sai is

is good

hanuman Sai

Sai is
 is coming

unique Bigrams = 08

$10 - 2 = 8 \checkmark$ (combs)

It looks like the boy

new rest of the thing is same;

Bag of Bi-grams:-

my name | name is | is sai | sai hamman | sai is | is good | hamman dai | is coming

s_1	1	1	1	1	0	0	0	0
s_2	0	0	0	0	1	1	0	0
s_3	0	0	0	0	1	0	1	1

Finally Bag of Bi-grams for s_1, s_2, s_3

$$s_1 = 11110000$$

$$s_2 = 00001100$$

$$s_3 = 00001011$$

② digit vector { because Bag of Bi-grams is 8 } ✓

OHE

BOW

BON-gram (or) N-grams feature selection

TF-IDF

Basic vectorization approaches;

Ex Pm TF-IDF

s_1	dog bites man
s_2	man bites dog
s_3	dog eats meat
s_4	man eats food

word	TF	IDF	TF-IDF score
dog	$\frac{1}{3} = 0.33$	$\log_e(4/3)$	0.36
bites	$\frac{1}{6}$	$\log_e(4/2)$	
man	$\frac{1}{3}$	$\log_e(4/3)$	
eats	$\frac{1}{6}$	$\log_e(2)$	
meat	$\frac{1}{12}$	$\log_e(4)$	
food	$\frac{1}{12}$	$\log_e(4/1)$	

test
(51) - (52) ✓

TF-IDF = Take a vocabulary, i.e. = 7

(ii) starts encoding

here

means



$$\frac{\text{count of term (t) in doc (d)}}{\text{Total words in doc (d)}}$$



(what is the prob. of picking (my) from the current sentence)

$$\log_e \left(\frac{\text{Total no. of sentences}}{\text{sentences containing (my)}} \right)$$

so for

$$S_1 \Rightarrow \text{my} \rightarrow TF = 1/6 \quad \text{IDF} = \log_e \left(\frac{3}{1} \right) \Rightarrow 0.26$$

$$\text{name} \Rightarrow TF = 1/6 \times \log_e \left(\frac{3}{1} \right) = 0.26$$

$$\text{is} \Rightarrow TF = 1/6 \times 0, \log_e \left(\frac{3}{3} \right) = 0$$

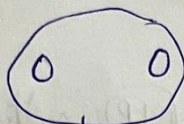
⋮

my name is hai hanuman good coming

S₁

0.26

0.26



0.098



for the words which not presents in the sentences;

for the words which presents in (all) the sentences;

S₂

0

0

0

0

0

0.528

0

S₃

0

0

0

0

0.146

0

0.075

so finally

(S₁)

vector

$$\Rightarrow [0.26 \quad 0.26 \quad 0 \quad 0 \quad 0.98 \quad 0 \quad 0]$$

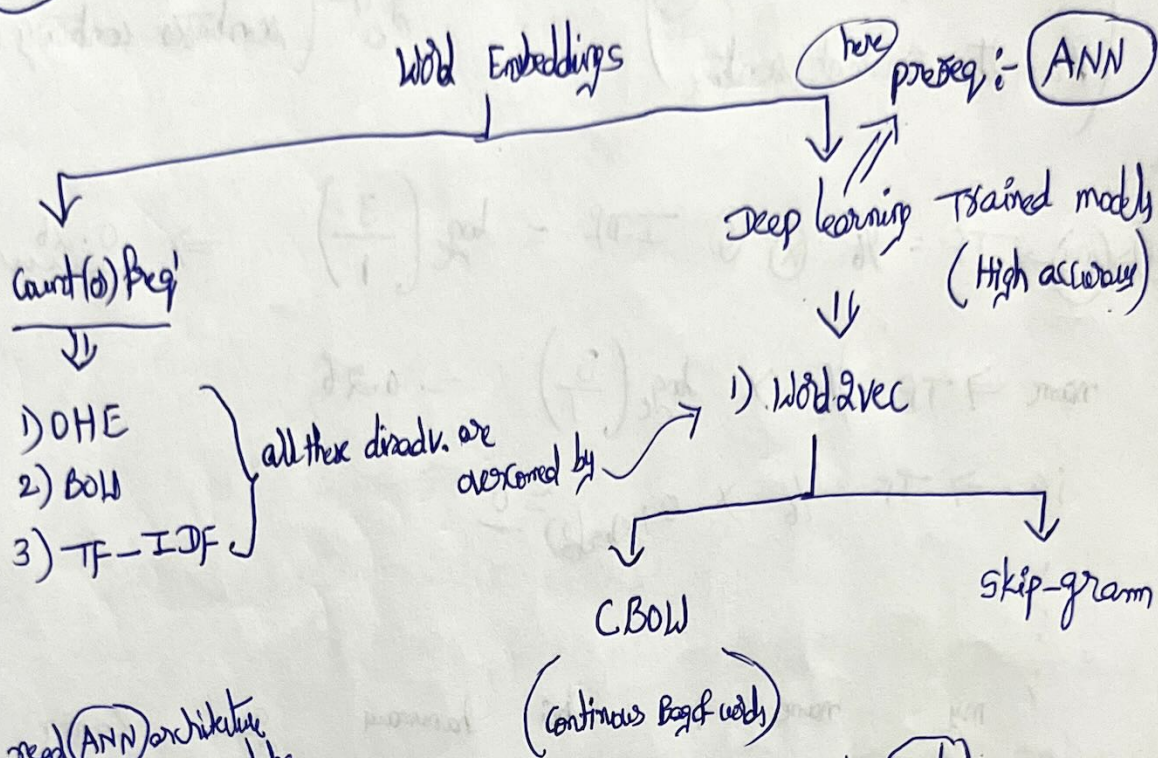
$$S_2 \Rightarrow [0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0.528 \quad 0]$$

$$S_3 \Rightarrow [0 \quad 0 \quad 0 \quad 0 \quad 0.146 \quad 0 \quad 0.075] \checkmark$$

we will get uniform size of length 7;

* Word Embedding :- (wikipedia)

In NLP, WE is a term used for the repⁿ of words for text analysis, typically in form of a real valued vector that encodes the meaning of the word such that the words that are close in the vector space are expected to be similar in meaning.



read ANN architecture knowledge

* Word2vec :- It is a technique for NLP published in 2013;

- this algorithm uses a neural network model to learn word associations from a large corpus of text;

- once trained, such model can detect synonyms words & suggest additional words for a partial sentence;

- As the name implies, Word2vec rep's each distinct word with a particular list of numbers called a vector; i.e. at the end text $\rightarrow$ vector

* Vocabulary = unique words (in corpus)

* Feature Representation

has many things to detect like synonyms
antonyms
add words

⊗ vocabulary $\rightarrow$ unique words $\rightarrow$ corpus

⊗ $v = \{ \text{Boy, girl, king, queen, apple, mango} \}$

Feature $\rightarrow$

	Boy	girl	king	queen	apple	mango
gender	-1	1	-0.92	+0.93	0.01	0.02
royals	0.01	0.02	0.95	0.96	-0.02	+0.02
age	0.03	0.02	0.75	0.68	0.98	0.95
food	:	:	:	:	0.91	0.92
:	:	:	:	:	:	:
n th	:	:	:	:	:	:

Feature Representation

300 dimensions

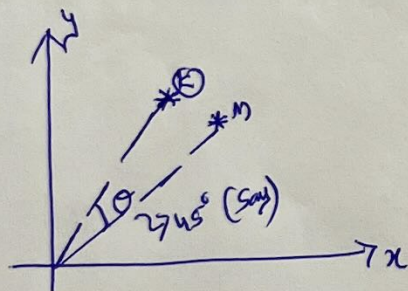
other values comes from model training;

(Google trained 3B words)

$\text{king} - \text{Man} + \text{Queen} = \text{Woman}$ very popular WE by Google ✓

like $\text{king} - \text{Boy} + \text{Queen} = \text{girl}$

⊗ say $\odot$: $\text{king} [0.95 \ 0.96] \quad \text{man} [0.95 \ 0.98] \quad \text{Queen} [-0.96 \ 0.95]$
 $\text{woman} [-0.94 \ -0.96]$



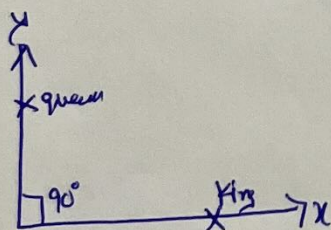
distance b/w k & m is '1' from Cosine similarity concept

$$\text{distance} = 1 - \text{Cosine similarity}$$

$$= 1 - \cos 45^\circ$$

$$= 1 - \frac{1}{\sqrt{2}} = 0.3 \text{ (say)}$$

that means they are close to each other



$$\text{distance} = 1 - \cos 90^\circ$$

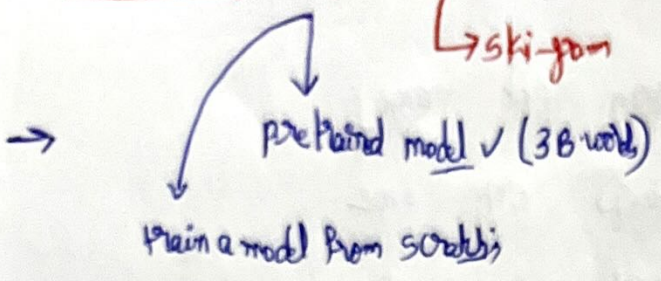
$$= 1 - 0$$

= 1 ✓ entirely different ✓

⊗ this concept is also used in Recommendation system ✓

Now we will see :- word2vec $\rightarrow$ CBOW $\rightarrow$ skip-gram

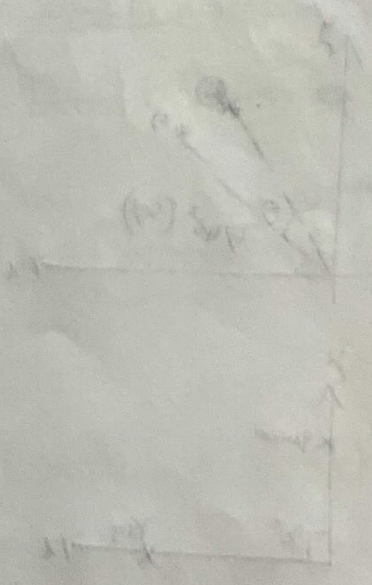
requirements :- ANN, loss func, optimizers;



① CBOW :- (Continuous Bag of Words) :- Nothing but fully connected NN;
Corpus

$$model = input + output - bias$$

$$loss = model - target$$



Handwritten signature or mark at the bottom right corner.